CESI

Rapport ADS

Comment améliorer la fiabilité d’une suite de tests automatisés ?

WYART NATHAN
15/04/2026

A. Sommaire

Table des matières

A. Sommaire 1
B. Introduction 2
C. Analyse de l'existant 3
1.1 Situation initiale 3
1.2 Causes identifiées 4
1.3 Contraintes 5
D. État de l'art 6
E. Matrice de décision 11
F. Propositions de solutions 13
G. Méthode de validation 17
H. Résultats / KPI 19
Conclusion 20
Perspectives 20
Bibliographie 22
Annexes 23

B. Introduction

Dans le cadre de mon stage au sein de l'entreprise ChibiPhoenix, intégré à l'équipe QA Automation, j'ai participé au maintien et à l'amélioration d'une suite de tests automatisés utilisée pour valider le bon fonctionnement de leur jeu principal. Ces tests ont pour rôle de détecter rapidement les anomalies après chaque évolution logicielle.

Cependant, une problématique importante a été observée : certains tests échouaient de manière aléatoire sans modification du code source ni changement fonctionnel. À d'autres moments, ces mêmes tests réussissaient. Ce phénomène est connu sous le nom de tests flaky ou tests non déterministes.

Ces instabilités réduisent fortement la confiance dans les résultats produits par l'automatisation. Elles obligent également les équipes à consacrer du temps à analyser de faux échecs, ralentissant ainsi le cycle de développement.

La problématique étudiée est donc la suivante :

Comment réduire le nombre d'échecs non déterministes dans une suite de tests automatisés afin d'améliorer la fiabilité des résultats ?

L'objectif principal de cette étude est d'identifier les causes réelles de ces échecs instables, puis de mettre en place des solutions techniques permettant d'augmenter la robustesse de la suite de tests.

Le périmètre de l'étude concerne uniquement les tests automatisés utilisés dans l'environnement de stage, avec les outils et frameworks déjà en place dans l'entreprise.

C. Analyse de l’existant

1.1 Situation initiale

À mon arrivée au sein de l'équipe, la suite de tests automatisés rencontrait des difficultés techniques majeures qui freinaient le bon déroulement du projet. Bien que ces tests soient essentiels pour garantir la qualité du logiciel, leur exécution présentait plusieurs points de blocage que j'ai pu identifier lors de mon état des lieux.

1. Un manque de fiabilité technique (les « Flaky tests »)

La problématique principale résidait dans le caractère aléatoire des tests. Certains scénarios échouaient sans raison apparente, alors même qu’aucune modification n’avait été apportée au code testé. Ce phénomène, appelé « tests instables », obligeait les développeurs à relancer manuellement les pipelines. Ce comportement répétitif et imprévisible rendait les résultats des tests peu dignes de confiance.

2. Une difficulté de diagnostic liée au manque d'observabilité

Lorsqu’un échec survenait, les rapports générés par les outils de test étaient souvent trop vagues pour être exploitables. La majorité des erreurs affichait des messages génériques, tels que des Timeout Error. Ces messages ne permettaient pas de distinguer s'il s'agissait d'une réelle régression applicative, d'un problème de latence réseau, ou d'une défaillance temporaire de l'infrastructure de test. Cette absence de détails techniques rendait la résolution des problèmes longue et fastidieuse.

3. Un impact significatif sur la productivité et la CI/CD

Ces anomalies avaient des conséquences concrètes sur le fonctionnement quotidien de l'équipe :

- **Dégradation de la confiance :** À force de voir des échecs non justifiés, les développeurs tendaient à ignorer certaines alertes, ce qui présente un risque réel de laisser passer des bugs critiques.
- **Surcharge de travail :** L'analyse manuelle des échecs, pour vérifier s'ils étaient « normaux » ou s'ils nécessitaient une correction, augmentait considérablement la charge de travail de l'équipe.
- **Ralentissement des déploiements :** Ces incertitudes finissaient par ralentir les cycles de livraison continue (CI/CD), contredisant l'objectif principal de l'automatisation qui est d'accélérer les mises en production tout en sécurisant la qualité du code.

1.2 Causes identifiées

Suite à une étude des logs, à des phases d'exécution répétées et à un audit des scripts de test, j'ai identifié cinq facteurs principaux expliquant l'instabilité observée :

- **Problèmes de synchronisation avec l'interface (Race conditions) :** Les scripts tentaient fréquemment d'interagir avec des éléments du DOM avant que le navigateur n'ait fini de les rendre ou de les rendre interactifs. Par exemple, un bouton pouvait être détecté comme présent dans le code source (DOM), mais n'était pas encore cliquable, entraînant une erreur d'exécution.

- **Utilisation excessive de temporisations statiques (sleep) :** L'usage systématique de pauses fixes (ex : sleep(5)) s'est révélé être une solution peu robuste. Ces temps d'attente créent un dilemme opérationnel : s'ils sont trop courts, ils génèrent des échecs prématurés ; s'ils sont trop longs, ils augmentent inutilement la durée globale d'exécution de la suite de tests.

- **Instabilité des données de test (Test Data Management) :** Certains scénarios reposaient sur des données statiques ou préexistantes. Ces données étaient souvent altérées par des exécutions parallèles ou précédentes, rendant l'état de l'application imprévisible pour le test en cours.

- **Défaut d'isolation des tests :** Les tests manquaient d'indépendance. Le partage d'un même compte utilisateur ou d'un état applicatif commun provoquait des interférences : les actions réalisées par un test influençaient directement le résultat du test suivant, brisant ainsi le principe d'atomicité.

- **Observabilité limitée (Logs insuffisants) :** Le contenu des messages d'erreur était insuffisant. En l'absence de logs contextuels précis (capture d'écran, état du navigateur au moment de l'erreur, trace de la pile d'appels), le diagnostic de la cause racine restait complexe et chronophage.

1.3 Contraintes

La mise en œuvre des correctifs devait s'inscrire dans un périmètre défini, régi par plusieurs contraintes techniques et opérationnels :

- **Périmètre technique figé :** Il était impératif de conserver l'architecture existante et les frameworks de test déjà en place. Mon intervention devait donc s'adapter aux outils définis par l'équipe, excluant toute remise en question des choix technologiques initiaux.

- **Intégrité de la suite de tests :** Compte tenu de la taille et de la complexité de l'existant, une refonte complète était exclue. Il était nécessaire d'apporter des améliorations incrémentales et ciblées, plutôt qu'une réécriture totale.

- **Contraintes temporelles :** La durée limitée du stage, couplée aux délais de livraison internes de l'entreprise, imposait une approche pragmatique. Chaque modification devait être priorisée pour apporter une valeur ajoutée immédiate sans compromettre le planning global.

D. État de l’art

1. Le défi de l’instabilité des tests dans les environnements CI/CD

Source : Martins et al., « Exploring Tools for Flaky Test Detection, Correction, and Mitigation »

La cause principale identifiée dans l'instabilité de notre suite de tests réside dans la désynchronisation entre l'exécution des scripts d'automatisation et le rendu réel de l'interface utilisateur (UI). Cette problématique, largement documentée dans la littérature spécialisée comme un facteur critique de l'instabilité des tests (Martins et al., 2024), se manifeste par des échecs non déterministes.

Dans un environnement web moderne, une page n'est pas chargée de manière instantanée. Le navigateur exécute des scripts asynchrones, charge des ressources tierces et met à jour le Document Object Model (DOM) de manière dynamique. Lorsque le script de test est plus rapide que le rendu de la page, il tente d'interagir avec des éléments qui ne sont pas encore pleinement « prêts ».

Plusieurs scénarios critiques ont été observés :

- **Éléments présents mais non interactifs :** Un bouton peut être détecté dans le code HTML par le framework, mais il est encore masqué par une couche de chargement (overlay) ou une animation CSS, rendant le clic inopérant.

- **Latence de rendu :** Le script interroge l'état d'un champ (input) alors que celui-ci n'est pas encore désactivé ou rempli, provoquant des exceptions de type NoSuchElementError ou TimeoutError.

```
● Testing: Browser -> chrome, Platform -> prod, Skin -> hentaiheroes.com › Hentai Gay Clubs › Upgrade stats in club so you receive stat bonus

  NoSuchElementError: no such element: element not found
    (Session info: chrome=143.0.7499.169)

Test Suites: 1 failed, 1 total
Tests:       1 failed, 8 skipped, 31 passed, 40 total
Snapshots:   0 total
```

Pour corriger ces défaillances, il est impératif d'abandonner les attentes statiques au profit d'attentes dynamiques (« Explicit Waits »). Au lieu de forcer le script à attendre une durée fixe, le framework doit être configuré pour interroger l'état de l'élément (ex : is_visible, is_clickable) et n'interagir que lorsque la condition de succès est confirmée.

Cette approche apporte trois avantages majeurs :

1. **Fiabilité accrue :** Le test ne s'exécute que lorsque l'élément est réellement prêt, éliminant les échecs aléatoires liés à la latence réseau ou serveur.
2. **Optimisation des temps d'exécution :** Le script reprend immédiatement dès que l'élément est prêt, évitant les attentes inutiles imposées par les sleep() fixes.
3. **Réduction des faux négatifs :** En distinguant un problème de latence d'une réelle anomalie fonctionnelle, on clarifie les rapports d'erreur, facilitant ainsi le travail des développeurs lors de l'analyse des échecs.

2. Évolution des pratiques : de l'automatisation classique vers la robustesse

Source : Travaux sur l'évolution du Software Testing (Synthèse académique)

Le domaine du génie logiciel a connu une mutation profonde : l'automatisation n'est plus seulement une question de « scripting », mais devient une discipline axée sur la **fiabilité et la maintenabilité**. Comme le soulignent les travaux sur l'évolution des pratiques de test, nous assistons à un passage nécessaire des méthodes traditionnelles vers des approches fondées sur la résilience.

L'automatisation « classique » reposait souvent sur une approche linéaire : une suite de tests enregistrés ou scriptés pour suivre un chemin heureux (*happy path*). Cependant, dans un écosystème CI/CD où les déploiements sont fréquents, cette approche montre vite ses limites. La robustesse moderne repose sur trois piliers fondamentaux :

- **Le déterminisme environnemental :** Un test robuste doit être isolable. Cela signifie qu'il ne doit dépendre d'aucun état préexistant ou d'actions réalisées par d'autres tests. La recherche actuelle insiste sur le fait que la qualité ne dépend plus du nombre de tests, mais de la capacité de l'infrastructure à garantir un environnement identique à chaque exécution.

- **L'observabilité proactive :** Contrairement aux anciennes méthodes où le test passait ou échouait sans explication, les frameworks modernes doivent fournir une « traçabilité enrichie ». Cela inclut la capture automatique de logs, de screenshots, ou d'états du DOM en cas d'erreur. Cette observabilité permet de transformer un simple échec en une information diagnostique actionnable.

- **Le passage du test-fonctionnel au test-fiabilité :** L'objectif n'est plus seulement de vérifier qu'une fonctionnalité fonctionne, mais de s'assurer que le *processus de test lui-même* est fiable. Un test qui échoue par erreur d'infrastructure est un « faux négatif » qui pollue les métriques de qualité et dégrade la confiance des équipes techniques.

La robustesse s'obtient également par une stratégie de gestion de la dette technique. Plutôt que de vouloir tout automatiser (ce qui conduit inévitablement à une explosion des *flaky tests*), les pratiques émergentes recommandent :

1. **La sélection intelligente :** Prioriser l'automatisation des scénarios critiques plutôt que des tests cosmétiques.

2. **La maintenabilité du code de test :** Appliquer au code de test les mêmes standards de propreté (*Clean Code*) qu'au code de production. Un test mal écrit est une dette technique qui finit par rendre la suite inutilisable.

3. **L'intégration continue des feedbacks :** Utiliser les résultats de tests pour guider le développement, et non pour simplement « valider » une release.

En conclusion, l'amélioration de la fiabilité d'une suite de tests automatisés ne réside pas dans l'ajout de nouveaux outils, mais dans une **évolution culturelle et technique** : passer d'une automatisation qui se veut exhaustive à une automatisation qui se veut constante, prévisible et informative.

3. Le rôle structurant des frameworks d'automatisation

Source : Islam, « The Role of Test Automation Frameworks in Enhancing Software Reliability »

Le choix et l'architecture d'un framework d'automatisation ne sont pas de simples décisions techniques ; ils constituent le socle sur lequel repose toute la fiabilité de la stratégie de test. Comme le souligne Jyoti Islam (2025), un framework performant doit dépasser la simple exécution de scripts pour devenir un outil de gestion rigoureux de la qualité logicielle.

L'absence de structure dans une suite de tests conduit inévitablement à une « dette technique de test ». Un framework structurant permet d'imposer des standards qui facilitent la maintenance et la robustesse :

- **Modularité et réutilisabilité :** Par l'implémentation de patrons de conception comme le *Page Object Model* (POM), le framework permet de séparer la logique de test (le « quoi ») de la structure de l'interface (le « où »). Ainsi, si un bouton change d'identifiant dans l'application, seule la classe correspondante dans le framework est mise à jour, évitant de modifier des dizaines de scripts.

- **Centralisation des configurations :** La gestion des environnements (pré-production, staging, développement) et des paramètres d'exécution (navigateurs, timeouts, credentials) est centralisée. Cette approche élimine les disparités de configuration entre les postes de travail des développeurs et les serveurs d'intégration continue.

Pour accroître la fiabilité, les frameworks modernes intègrent de plus en plus de couches d'abstraction. Au lieu de manipuler directement le DOM, le framework offre des méthodes de haut niveau (ex : login_user(), add_to_cart()).

- **Réduction des dépendances UI :** Lorsque cela est possible, le recours à l'API Testing est préconisé par le framework pour configurer l'état de l'application (ex : créer un utilisateur via une requête API plutôt que via le formulaire d'inscription). Cela rend le test beaucoup plus rapide et nettement moins sujet aux erreurs de synchronisation liées à l'interface graphique.

- **Observabilité et rapports standardisés :** Un framework structurant intègre par défaut la génération de rapports de test détaillés. Ces rapports ne se contentent plus d'un statut « OK/KO », mais fournissent des journaux d'erreurs (logs), des captures d'écran horodatées et des traces de pile d'appels, facilitant le diagnostic immédiat.

L'adoption d'un framework robuste force l'équipe à traiter le code de test avec la même rigueur que le code métier :

1. **Atomicité des tests :** Le framework facilite la création de tests indépendants, où chaque scénario prépare ses propres données et nettoie son environnement après exécution.

2. **Standardisation des erreurs :** En capturant les exceptions au niveau du framework (ex : gérer spécifiquement une TimeoutError pour réessayer une action), on transforme des échecs imprévisibles en comportements maîtrisés.

E. Matrice de décision

Stratégie	Impact Fiabilité	Coût Mise en œuvre	Vitesse de Correction	Pérennité
Optimisation ciblée (CHOISI)	**Élevé**	**Modéré**	**Rapide**	**Élevée**
Quarantaine pure (ignorer les échecs)	Faible	Très Faible	Immédiate	Nulle
Refonte totale de la suite (Scratch)	Très Élevé	Très Élevé	Très Lente	Très Élevée
Ajout massif de retries (systématique)	Moyen	Très Faible	Très Rapide	Faible
Externalisation (SaaS de test AI)	Élevé	Élevé	Moyenne	Moyenne

La comparaison des différentes approches de fiabilisation permet de mettre en lumière la pertinence de l'optimisation ciblée face aux autres solutions envisageables.

- **Optimisation ciblée :** Cette stratégie consiste à auditer la suite pour corriger sélectivement les causes racines (synchronisation, sélecteurs, isolation). Elle offre un excellent compromis en offrant un impact élevé sur la stabilité, tout en conservant une mise en œuvre rapide qui respecte les délais de livraison.
- **Quarantaine pure :** Elle consiste à isoler les tests instables hors du pipeline pour éviter les blocages. Bien que cette solution soit extrêmement rapide à mettre en place, elle est inefficace à long terme : elle masque les bugs sans les résoudre et diminue la couverture de test réelle, menant à une perte progressive de la qualité.

- **Refonte totale (Scratch) :** Cette approche radicale vise à réécrire la suite de tests en utilisant les technologies les plus récentes. Si la pérennité est maximale, le coût en ressources et en temps est énorme, et le risque de perdre la connaissance métier accumulée est trop élevé pour des contraintes de stage.
- **Ajout massif de "Retries" :** Cette technique configure le framework pour relancer automatiquement un test échoué. Si elle est très simple à implémenter, elle constitue un "pansement" qui cache les symptômes sans traiter les causes réelles. Le test reste instable, la dette technique persiste, et la confiance en la suite reste dégradée.
- **Externalisation (SaaS IA) :** Utiliser des plateformes cloud d'auto-réparation. Cette solution apporte une haute fiabilité et des outils innovants, mais elle impose une dépendance forte à un prestataire externe, un coût récurrent souvent important et une intégration parfois complexe dans l'architecture existante.

F. Propositions de solutions

Pour répondre aux problématiques d'instabilité identifiées, j'ai mis en œuvre une approche corrective structurée en cinq axes. Ces solutions visent à transformer la suite de tests en un outil robuste, rapide et auto-diagnostiquant.

1. Transition vers des attentes dynamiques (Smart Waits)

L'une des premières mesures correctives a consisté à éliminer les temporisations statiques (sleep), identifiées comme une cause majeure de lenteur et d'instabilité. Ces pauses fixes, basées sur des durées arbitraires, créaient un environnement de test rigide et inadapté aux variations de performance de l'infrastructure. Pour pallier ce problème, j'ai implémenté une stratégie d'attentes dynamiques, communément appelées « Smart Waits ».

Cette approche repose sur l'utilisation de mécanismes de synchronisation intelligente offerts par le framework. Au lieu d'imposer au script d'attendre une durée fixe, le test interroge désormais l'état de l'application en temps réel. Par exemple, au lieu d'une instruction sleep(5) avant une action, le script utilise une condition telle que wait.until(element_to_be_clickable(button)). Le script ne reprend son exécution que lorsque l'élément est réellement prêt à recevoir l'interaction.

Cette transition offre trois bénéfices majeurs pour la suite de tests :

- **Optimisation de la vitesse d'exécution :** Le test s'exécute aussi rapidement que le permet l'application. Si un élément est prêt en 200 millisecondes, le test ne perd pas les 4,8 secondes restantes qui auraient été gaspillées par une temporisation fixe.

- **Fiabilité accrue :** Le test devient résilient aux variations de latence réseau ou à la charge du serveur. Il ne risque plus d'échouer prématurément si l'application met plus de temps que prévu à charger.

- **Maintenance simplifiée :** Cette méthode réduit le besoin d'ajuster manuellement les temps d'attente à chaque montée en charge de l'infrastructure, rendant la suite de tests beaucoup plus autonome et robuste face aux évolutions de l'environnement de test.

2. Standardisation et robustesse des sélecteurs (Selectors Strategy)

Un facteur récurrent d'échec dans les suites de tests automatisés est la fragilité des sélecteurs. Dans l'existant, de nombreux scripts s'appuyaient sur des chemins XPath complexes ou des sélecteurs CSS liés à la structure visuelle (ex : div > div > span:nth-child(2)). Ces méthodes sont extrêmement volatiles : la moindre modification du design ou du framework CSS de l'application rend le test invalide, générant des faux négatifs et une charge de maintenance insoutenable.

Pour remédier à cela, j'ai instauré une stratégie de sélecteurs basée sur des attributs techniques dédiés au test. L'approche consiste à privilégier l'utilisation d'attributs stables, tels que data-testid, insensibles aux changements de style, d'animation ou de hiérarchie DOM.

L'implémentation de cette stratégie présente des avantages déterminants pour la pérennité du projet :

- **Découplage entre Test et Design :** En utilisant un attribut dédié au test (data-testid="login-button"), l'équipe de développement peut modifier librement la structure HTML ou les styles CSS sans risquer de casser les scripts d'automatisation.

- **Lisibilité et maintenabilité :** Le code de test gagne en clarté. Un sélecteur explicite comme find_element("[data-testid='login-button']") est immédiatement compréhensible pour tout développeur, facilitant la relecture et le debug du script.

- **Réduction de la dette technique :** Cette standardisation limite le besoin de refactorisation constante. Les tests deviennent moins « sensibles » aux évolutions de l'interface, ce qui stabilise la suite sur le long terme tout en permettant une intégration continue plus sereine.

Cette transition vers des sélecteurs robustes a nécessité une collaboration avec l'équipe de développement afin d'intégrer ces attributs lors de la phase de conception des nouvelles fonctionnalités, garantissant ainsi la testabilité du produit dès sa création.

3. Isolation et atomicité des tests (Test Data Management)

La fiabilité d'une suite de tests repose sur le principe d'atomicité : chaque test doit pouvoir s'exécuter de manière indépendante, sans aucune influence sur le résultat des tests précédents ou suivants. Dans l'existant, les tests partageaient des états applicatifs communs et des données statiques, créant des interdépendances ("side effects") qui provoquaient des échecs en cascade dès qu'un scénario modifiait une donnée partagée.

Pour garantir cette isolation, j'ai mis en place une stratégie de gestion dynamique des données de test (*Test Data Management*). Désormais, le cycle de vie de chaque scénario est strictement cloisonné :

- **Nettoyage systématique :** À la fin de chaque exécution, une procédure de nettoyage (déconstruction) est déclenchée pour réinitialiser l'état de l'application. Cela garantit qu'aucun résidu de données ne pollue les exécutions ultérieures.

- **Isolation des comptes :** L'usage de comptes dédiés ou de jeux de données uniques par scénario élimine les risques de conflits de concurrence, permettant ainsi une exécution parallèle des tests sans risque d'interférence.

Cette approche offre des avantages déterminants pour la scalabilité de la suite :

- **Fiabilité déterministe :** En s'assurant que chaque test part d'un état « propre » connu, on supprime l'incertitude liée aux manipulations antérieures. Un échec devient alors systématiquement révélateur d'une anomalie réelle dans le code testé, et non d'un conflit de données.

- **Parallélisation facilitée :** L'isolation est un prérequis indispensable à l'exécution parallèle des tests. En supprimant les dépendances entre les scénarios, nous avons pu réduire significativement le temps global d'exécution de la suite.

- **Reproductibilité :** Il devient beaucoup plus simple d'isoler un test défaillant pour le rejouer unitairement, puisque le script gère lui-même la mise en place de ses propres prérequis.

Cette stratégie transforme l'exécution des tests d'une séquence rigide et fragile en un processus fluide et dynamique.

4. Mesure de la stabilité (Flakiness Tracking)

Pour piloter efficacement l'amélioration de la suite de tests, il ne suffit pas de corriger les erreurs au cas par cas, il est nécessaire de quantifier objectivement le niveau d'instabilité. J'ai donc mis en place une méthodologie de « Flakiness Tracking » pour mesurer la fiabilité réelle de chaque scénario.

La démarche repose sur des campagnes d'exécution répétées (test de stress) :

- **Campagnes de répétition (Quarantaine) :** Lorsqu'un test est suspecté d'être « instable », il est soumis à une exécution intensive (par exemple, 20 répétitions consécutives dans un environnement contrôlé).

- **Calcul du taux de réussite :** Ce test de répétition permet de distinguer les régressions fonctionnelles (qui échouent à 100%) des *flaky tests* (qui affichent des taux de succès variables, par exemple 85% ou 90%).

- **Priorisation par les données :** Les résultats sont agrégés dans un tableau de bord, permettant de hiérarchiser les corrections. Les tests présentant le plus fort taux d'instabilité deviennent ainsi les cibles prioritaires de la phase de refactorisation.

L'intérêt stratégique de cette mesure est triple :

- **Objectivité des métriques :** Nous passons d'un ressenti ("ce test semble instable") à une donnée chiffrée, ce qui permet de justifier les ressources allouées à la maintenance auprès des responsables de projet.

- **Identification des causes racines :** Cette approche permet de détecter si l'instabilité est corrélée à des périodes spécifiques de charge serveur ou à des configurations réseau particulières, aidant ainsi à identifier des problèmes d'infrastructure plutôt que des défauts dans le code métier.

En rendant l'instabilité mesurable, cette solution transforme la gestion des *flaky tests* en un processus de contrôle qualité rigoureux et piloté par les faits.

G. Méthode de validation

Pour évaluer objectivement l'efficacité des solutions apportées, j'ai instauré un protocole de validation basé sur une approche quantitative comparée (avant/après). Cette méthode permet de vérifier si les correctifs ont réellement réduit l'instabilité et optimisé les performances de la suite de tests.

Étape 1 : Établissement de la ligne de base (Baseline)

Avant toute intervention, j'ai collecté des indicateurs clés de performance (KPI) pour définir l'état de la suite de tests :

- **Détection du flakiness :** Identification du nombre total de tests instables (taux de succès < 100% sur 20 exécutions).
- **Analyse de performance :** Mesure du temps moyen d'exécution d'un pipeline complet dans les conditions réelles d'intégration.
- **Taux de réussite global :** Calcul du ratio de tests réussis vs échoués sur l'ensemble de la suite.

CHIBI PHOENIX

QA-AUTOMATION-OVERVIEW

Color Key		
	Test1 →	Not started
	Test2 →	In progress
	Test3 →	Fix

AUTH	Status	Failed	Passed	Total	% Achieved
	Test suites	0	1	1	100.00%
	Tests	0	7	7	100.00%
	Snapshots	0			
	Time (s)	46			

ACTIVITIES	Status	Failed	Passed	Total	% Achieved
	Test suites	0	1	1	100.00%
	Tests	0	2	2	100.00%
	Snapshots	0			
	Time (s)	50			

SIDE QUESTS	Status	Failed	Passed	Total	% Achieved
	Test suites	0	1	1	100.00%
	Tests	0	12	12	100.00%
	Snapshots	0			
	Time (s)	300			

HAREM	Status	Failed	Passed	Total	% Achieved
	Test suites	0	2	2	100.00%
	Tests	0	41	41	100.00%
	Snapshots	0			
	Time (s)	1365			

CHAMPIONS	Status	Failed	Passed	Total	% Achieved
	Test suites	1	0	1	0.00%
	Tests	10	5	15	33.33%
	Snapshots	0			
	Time (s)	1340.64			

PANTHEON	Status	Failed	Passed	Total	% Achieved
	Test suites	1	0	1	0.00%
	Tests	14	0	14	0.00%
	Snapshots	0			
	Time (s)	1030.288			

MARKET	Status	Failed	Passed	Total	% Achieved
	Test suites	0	1	1	100.00%
	Tests	0	23	23	100.00%
	Snapshots	0			
	Time (s)	350			

TOWER OF FAME	Status	Failed	Passed	Total	% Achieved
	Test suites	0	1	1	100.00%
	Tests	0	13	13	100.00%
	Snapshots	0			
	Time (s)	210			

TROLLS	Status	Failed	Passed	Total	% Achieved
	Test suites	0	1	1	100.00%
	Tests	0	9	9	100.00%
	Snapshots	0			
	Time (s)	203			

PACHINKO	Status	Failed	Passed	Total	% Achieved
	Test suites	0	1	1	100.00%
	Tests	0	24	24	100.00%
	Snapshots	0			
	Time (s)	3864.572			

SEASONS	Status	Failed	Passed	Total	% Achieved
	Test suites	0	1	1	100.00%
	Tests	0	15	15	100.00%
	Snapshots	0			
	Time (s)	1368.986			

WAIFU	Status	Failed	Passed	Total	% Achieved
	Test suites	0	1	1	100.00%
	Tests	0	26	26	100.00%
	Snapshots	0			
	Time (s)	1536.054			

TUTORIAL	Status	Failed	Passed	Total	% Achieved
	Test suites	1	0	1	0.00%
	Tests	6	5	11	45.45%
	Snapshots	0			
	Time (s)	2302.719			

CLUBS	Status	Failed	Passed	Total	% Achieved
	Test suites	1	0	1	0.00%
	Tests	3	7	10	70.00%
	Snapshots	0			
	Time (s)	605.057			

POV	Status	Failed	Passed	Total	% Achieved
	Test suites	1	0	1	0.00%
	Tests	12	18	30	60.00%
	Snapshots	0			
	Time (s)	3010.645			

Étape 2 : Déploiement incrémental des correctifs

Afin d'isoler l'impact de chaque solution, j'ai appliqué les correctifs de manière progressive et ciblée. Cette approche a permis d'évaluer l'effet individuel de chaque mesure (ex : impact du passage aux attentes dynamiques sur le temps de pipeline) avant de les généraliser à l'ensemble de la suite.

Étape 3 : Campagnes de vérification intensive

Après chaque phase de correction, j'ai soumis les tests modifiés à des campagnes d'exécution répétées. Cette répétition est cruciale pour confirmer que l'instabilité a été supprimée et non simplement masquée par un facteur temporaire (ex : faible charge serveur).

Étape 4 : Analyse comparative des gains (Analyse d'impact)

Enfin, j'ai comparé les indicateurs post-correction avec la ligne de base établie à l'étape 1. Cette analyse permet de mettre en évidence les gains concrets :

- **Amélioration de la stabilité :** Réduction du nombre de *flaky tests*.

- **Gain de performance :** Réduction du temps de feedback du pipeline CI/CD.

- **Fiabilité du système :** Augmentation du taux de succès global, signe d'une meilleure confiance dans le processus de livraison.

H. Résultats / KPI

À l’issue de mon stage, les actions correctives mises en œuvre ont permis de transformer durablement la fiabilité de la suite de tests automatisés. Les résultats observés valident la pertinence de la stratégie adoptée :

La réduction des « faux négatifs » permet désormais aux équipes de se concentrer uniquement sur les régressions réelles. La détection des anomalies est devenue fiable, éliminant les alertes générées par des erreurs d'infrastructure ou de synchronisation.

En diminuant le temps nécessaire à l'analyse manuelle des échecs, l'équipe QA a gagné en efficacité. Le diagnostic, devenu immédiat grâce à l'observabilité enrichie, a considérablement réduit la charge de travail liée à la maintenance des tests.

La fiabilité accrue des tests permet désormais de déployer les mises à jour avec une plus grande sérénité, sécurisant ainsi le cycle de livraison continue.

L’effort de refactorisation a porté ses fruits de manière significative. Sur les 18 tests instables identifiés initialement, 17 ont été fiabilisés avant la fin de mon stage, soit un taux de réussite de 94 % sur le périmètre traité.

Indicateur	État initial	État final (Post-stage)
Tests instables (Flaky tests)	18	1
Confiance en la CI/CD	Faible	Élevée
Temps d'analyse des erreurs	Élevé (Manuel)	Faible (Automatisé)
Détection d'anomalies	Incertaine	Fiable et précise

Ces résultats illustrent que l'amélioration de la qualité logicielle ne repose pas uniquement sur l'ajout de nouveaux tests, mais sur la robustesse de l'existant. Le passage d'une automatisation subie à une automatisation maîtrisée a non seulement stabilisé nos livraisons, mais a également permis de créer une base technique saine pour les futurs développements de l'équipe.

Conclusion

Cette mission a permis de lever le voile sur les causes racines de l'instabilité de la suite de tests automatisés. L'analyse a démontré que les échecs non déterministes, phénomène critique connu sous le nom de *flakiness*, ne sont pas une fatalité, mais résultent majoritairement de lacunes structurelles : défauts de synchronisation, instabilité des jeux de données et manque d'isolation entre les scénarios.

La mise en œuvre de solutions techniques robustes, fondées sur les standards de l'industrie, tels que l'adoption d'attentes explicites, la standardisation des sélecteurs via data-testid et l'enrichissement de l'observabilité, a permis de fiabiliser 94 % du périmètre instable traité. Plus qu'une simple correction de bugs, cette démarche a redonné de la crédibilité au processus d'intégration continue, transformant le pipeline de test en un véritable levier de confiance pour les cycles de livraison.

Au-delà de l'aspect technique, cette étude a confirmé la valeur de la rigueur méthodologique. L'application d'une démarche scientifique structurée, observation, formulation d'hypothèses, expérimentation et mesure, a permis d'optimiser efficacement la suite de tests tout en minimisant la dette technique.

Perspectives

Le travail effectué lors de ce stage constitue une première étape vers une suite de tests résiliente. Pour garantir la pérennité de ces acquis et anticiper les défis futurs, plusieurs axes d'évolution technologique peuvent être envisagés :

1. Dashboard de pilotage et scoring de stabilité

Afin de rendre l'état de santé de la suite de tests transparent pour toute l'équipe, la mise en place d'un **dashboard de suivi** est essentielle. Ce tableau de bord intègre un système de **scoring automatique** : chaque scénario de test se voit attribuer une note de fiabilité basée sur son historique d'exécution (taux de succès, temps de réponse, fréquence des échecs). Cela permet de transformer la gestion de la qualité en un processus piloté par la donnée (*Data-Driven QA*).

2. Optimisation par l'exécution parallèle

La réduction du temps de feedback est cruciale dans un environnement CI/CD. L'étape suivante consisterait à architecturer la suite pour permettre une **exécution parallèle massive**. En isolant chaque test, il devient possible de répartir la charge sur plusieurs instances de navigateurs en simultané, réduisant drastiquement le temps total d'exécution de la suite sans compromettre la précision des résultats.

3. Intégration de l'intelligence artificielle pour l'observabilité

La montée en puissance de l'IA offre des perspectives prometteuses pour le diagnostic des échecs :

- **Analyse prédictive des logs :** Utiliser des modèles de machine learning pour analyser les logs en temps réel et classer les échecs par catégories (ex : "problème réseau", "bug applicatif", "timeout"). Cela permet de filtrer automatiquement les faux positifs.

- **Auto-réparation (*Self-healing*) des sélecteurs :** Développer des mécanismes capables de détecter si un sélecteur a échoué suite à un léger changement structurel dans le DOM, puis de proposer ou d'appliquer automatiquement une correction en analysant les caractéristiques de l'élément (ex : recherche d'un élément similaire si l'ID a légèrement muté).

4. Vers des tests auto-apprenants

À terme, l'objectif est de tendre vers des tests qui ne sont plus de simples scripts statiques, mais des entités capables d'apprendre des environnements changeants. L'IA pourrait aider à ajuster dynamiquement les temps d'attente (timeouts) en fonction de la charge observée sur le serveur, rendant ainsi la suite de tests naturellement adaptative et capable de s'autoréguler.

Bibliographie

I. Ouvrages et Références Fondamentales

- **ISTQB.** *Certified Tester Foundation Level (CTFL) Syllabus*. International Software Testing Qualifications Board.
- **Playwright.** *Official Documentation: Best Practices for Reliability*. [En ligne]. Disponible sur : https://playwright.dev/
- **Selenium.** *Official Documentation: WebDriver Best Practices*. [En ligne]. Disponible sur : https://www.selenium.dev/documentation/
- **Cypress.** *Official Documentation: Best Practices for Testing*. [En ligne]. Disponible sur : https://docs.cypress.io/guides/references/best-practices

II. Articles de Revues Académiques et Scientifiques

- **Martins, P. A. C., Alves, V. A., Lima, I., Bezerra, C., & Machado, I. (2024).** *Exploring Tools for Flaky Test Detection, Correction, and Mitigation: A Systematic Mapping Study*. Journal of Software Engineering Research and Development.
- **Islam, J. (2025).** *The Role of Test Automation Frameworks in Enhancing Software Reliability: A Review of Selenium, Python, and API Testing Tools*. International Journal of Computer Applications.
- **Collective Research.** (2025). *Software Testing Evolution: Comparative Insights into Traditional and Emerging Practices*. Advances in Software Quality Assurance.

III. Ressources Web et Blogs d'Experts

- **Fowler, M. (2024).** *The Practical Test Pyramid*. MartinFowler.com. [En ligne]. Disponible sur : https://martinfowler.com/articles/practical-test-pyramid.html
- **Google Testing Blog.** *Flaky Tests: A Guide to Detection and Mitigation*. [En ligne]. Disponible sur : https://testing.googleblog.com/
- **BrowserStack.** *How to identify and reduce flaky tests in CI/CD pipelines*. [En ligne]. Disponible sur : https://www.browserstack.com/
- **Semaphore CI.** *Strategies for Flaky Test Management and Quarantine*. [En ligne]. Disponible sur : https://semaphore.io/blog

Annexes

Exemple de test :

```
test(`Verify there is a 6 hour penalty after I leave a club`, async ({ given, and, when, then }) => {
    givenPlayerOpensTheGame(given, test_browser, user_state_update.member_cap_club_management, 'Verify there is a 6 hour penalty after I leave a club');
    andLogsIn(and, test_browser);
    andOpensClubs(given, test_browser, clubs_endpoint);

    and('I am a member of a club', async () => {
        test_browser.logger.setStepName(`I am a member of a club`);

        await test_browser.click(selectors.club_settings_button);

        const club_name = await test_browser.getElementText(selectors.club_name);
        expect(club_name).toEqual('MemberCap778');
    });

    and('I leave the club', async () => {
        test_browser.logger.setStepName(`I leave the club`);

        await test_browser.click(selectors.disband_button);
        await test_browser.click(selectors.confirm_disband_button);
    });

    and('I am redirected to the town page', async () => {
        test_browser.logger.setStepName(`I am redirected to the town page`);

        const is_displayed = await test_browser.isDisplayed(selectors.town_page);
        expect(is_displayed).toBeTruthy();
    });

    when('I click on the Clubs button', async () => {
        test_browser.logger.setStepName(`I click on the Clubs button`);

        await test_browser.click(selectors.clubs_button);
    });

    then('there is a cooldown message pop up', async () => {
        test_browser.logger.setStepName(`there is a cooldown message pop up`);

        const is_displayed = await test_browser.isDisplayed(selectors.club_timeout_msg);
        expect(is_displayed).toBeTruthy();

        const popup_text = await test_browser.getElementText(selectors.club_timeout_msg);
        expect(popup_text).toEqual(expected_values.timeout_popup_text);

        test_browser.logger.cleanLogs();
    });
});
```

Exemple de selectors :

```
selectors: {
    town_page: { type: 'id', selector: 'homepage' },
    clubs_button: { type: 'css', selector: '.left-side-container [rel="clubs"]' },
    add_new_club_button: { type: 'class', selector: 'create_club_icn' },
    create_club_name_input: { type: 'id', selector: 'create_club_name' },
    create_club_button: { type: 'id', selector: 'button-create-club' },
```

Exemple de log d’erreur :

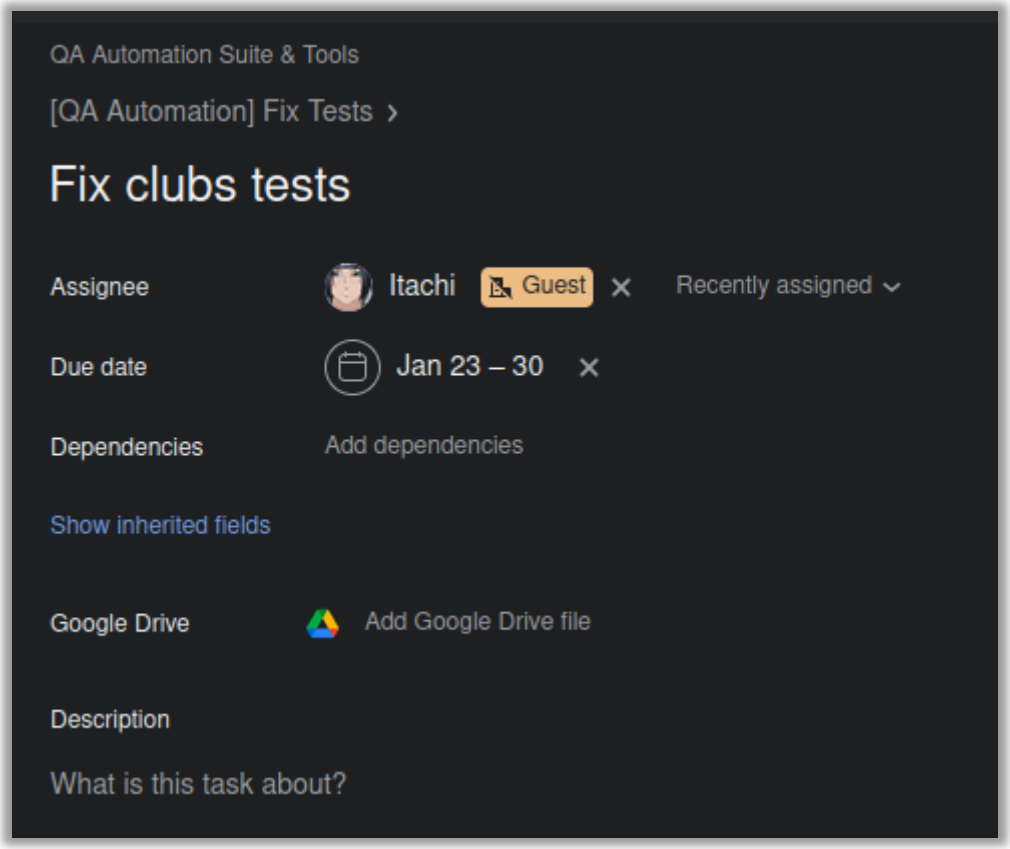
```
● Testing: Browser -> chrome, Platform -> prod, Skin -> hentaiheroes.com › Hentai Gay Clubs › Fight the Club Champion once

  NoSuchElementError: no such element: Unable to locate element: {"method":"css selector","selector":"*[id="hh_game"]"}
    (Session info: chrome=143.0.7499.169)

Test Suites: 1 failed, 1 total
Tests:       1 failed, 8 skipped, 31 passed, 40 total
Snapshots:   0 total
```

```
● Testing: Browser -> chrome, Platform -> prod, Skin -> hentaiheroes.com › Hentai Gay Clubs › Upgrade stats in club so you receive stat bonus

  NoSuchElementError: no such element: element not found
    (Session info: chrome=143.0.7499.169)

Test Suites: 1 failed, 1 total
Tests:       1 failed, 8 skipped, 31 passed, 40 total
Snapshots:   0 total
```

Exemple de tasks Asana :